Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Projektowanie efektywnych algorytmów

Implementacja i analiza efektywności algorytmu losowego, zachłannego - Nearest Neighour (NN) i Repetitive Nearest Neighbour (RNN) oraz Brute-Force (przegląd zupełny)

Wykonawca: Marta Witkowska (281222)

Prowadzący: Dr Inż. Jarosław Mierzwa

Data: 18.03.2026

WROCŁAW 2026

Spis treści

1 Wstęp **2**
1.1 Problem komiwojażera . . . 2
1.2 Algorytm Brute-Force . . . 2
1.2.1 Złożoność obliczeniowa . . . 2
1.3 Algorytm losowy . . . 2
1.3.1 Złożoność obliczeniowa . . . 2
1.4 Algorytm NN . . . 2
1.4.1 Złożoność obliczeniowa . . . 3
1.5 Algorytm RNN . . . 3
1.5.1 Złożoność obliczeniowa . . . 3

2 Przebieg eksperymentu **4**
2.1 Plan eksperymentu . . . 4
2.1.1 Rozmiar używanych struktur danych . . . 4
2.1.2 Sposób generowania danych . . . 4
2.1.3 Metoda pomiaru czasu . . . 4
2.1.4 Wpływ rodzaju danych na wyniki . . . 4
2.2 Wyniki eksperymentu . . . 5
2.2.1 Zależność czasowa algorytmu Brute Force . . . 5
2.2.2 Średni błąd względny algorytmów . . . 6
2.2.3 Średni czas algorytmów . . . 9

3 Podsumowanie **10**
3.1 Realizacja Projektu . . . 10
3.2 Wnioski . . . 10

4 Bibiliografia **11**

1. Wstęp

1.1 Problem komiwojażera

Problem komiwojażera polega na znalezieniu najkrótszej trasy między punktami, tak aby każdy z nich był odwiedzony tylko 1 raz. Na koniec komiwojażer musi wrócić do punktu startowego. Należy on do trudnych problemów, ponieważ wymaga on dużej dokłądności i prawdopodobnie nie istnieje taki algorytm, który mógłby znajdywać idealnej drogi dla dużych instancji.

1.2 Algorytm Brute-Force

Algorytm Brute-Force jest dokładnym algorytmem, ale bardzo powolnym. Sprawdza on wszystkie możliwe ścieżki i wybiera najkrótszą z nich.

1.2.1 Złożoność obliczeniowa

Złożoność obliczeniowa dla algorytmu sprawdzającego wszystkie opcje wynosi:

$$O((n-1)!) \tag{1.1}$$

1.3 Algorytm losowy

Algorytm losowy polega na wylosowaniu wielu możliwych dróg i wybranie najkrótszej z nich. Ze względu na swoją losowość ten algorytm jest niedokładny, droga którą wyznaczy ma małą szansę na bycie optymalną.
Aby zminimalizować ryzyko powtórzeń przy zachowaniu pełnej losowości, można użyć struktury zbioru (np. tablicy mieszającej) do zapamiętywania i odrzucania już wylosowanych permutacji.

1.3.1 Złożoność obliczeniowa

Algorytm wykona tyle zapętleń ile użytkownik będzie chciał mieć próbek. Na potrzeby tego projektu wykonywano $N * 10$ próbek, więc do analizy danych złożoność obliczeniowa wynosi:

$$O(n^2) \tag{1.2}$$

1.4 Algorytm NN

Algorytm Nearest Neighour (NN) polega na wybieranie najkrótszych dostępnych ścieżek z aktualnie dostępnego miejsca. Często algorytm wybiera na początku sto-

sunkowo krótką trasę, ale na końcu powrót do miejsca startu odbya się tymi "dłuższymi"ścieżkami. To rozwiązanie nie gwarantuje idealnego rozwiązania, ale stara się znaleźć wystarczająco dobre. Przez potrzebę znania własnej lokalizacji i odwiedzonych miejsc, algorytmy typu NN są bardzo zachłanne, ale nie jest złożony i działa bardzo szybko.

1.4.1 Złożoność obliczeniowa

Ilość operacji wykonywanych przy liczeniu danego algorytmu odpowiada sumie ciągu arytmetycznego, dlatego złożoność tego algorytmu wynosi:

$$O(n^2) \tag{1.3}$$

1.5 Algorytm RNN

Ze względu na to, że przy algorytmie NN miejsce startu ma naczenie, to algorytm Repetitive Nearest Neighbour (RNN) sprawdza dla wszystkich dostępnych punktów ścieżki algorytmem NN i wybiera najkrótszą z nich. Ten algorytm jest dokładniejszy od NN, ale dalej nie jest idealny. Tal samo jak algorytm NN jest zachłanny, ale bardzo szybko daje informację o stosunkowo krótkiej trasie.
Aby uzyskać dokładniejsze wyniki, algorytm można rozszerzyć o sprawdzanie rozgałęzień w miejscach gdzie niektóre ścieżki do różnych punktów mają tak samo niską wagę. Jeśli algorytm napotka z danego miasta kilka krawędzi o tej samej, najmniejszej wadze, musi zapamiętać ten węzeł i rekurencyjnie sprawdzić każdą z tych równorzędnych dróg. Dzięki temu rozwiązaniu nie przeoczy się innej, potencjalnie lepszej drogi.

1.5.1 Złożoność obliczeniowa

Algorytm RNN polega na użyniu n razy algorytmu NN, dlatego jego złożoność wynosi:

$$O(n^3) \tag{1.4}$$

2. Przebieg eksperymentu

2.1 Plan eksperymentu

2.1.1 Rozmiar używanych struktur danych

Badania przeprowadzono dla problemu komiwojażera o rozmiarach N wynoszących 10, 11, 12 oraz 13 wierzchołków (miast). Dla algorytmu Brute Force badania były przeprowadzane na wierzchołkach z zakresu od N=6 do N=14. Do reprezentacji grafu w pamięci komputera wykorzystano macierz sąsiedztwa, czyli dwuwymiarową tablicę o wymiarach N x N. Komórki tej macierzy przechowują wagi krawędzi między poszczególnymi miastami.

2.1.2 Sposób generowania danych

Dla każdego rozmiaru N wygenerowano 100 niezależnych, losowych instancji problemu (reprezentujących grafy pełne, gdzie każde miasto jest połączone z każdym innym). Odległości między miastami były losowane z ustalonego przedziału liczbowego liczb całkowitych dodatnich. Na przekątnej generowanych macierzy były ustawiane wartości -1.

2.1.3 Metoda pomiaru czasu

Czas wykonania poszczególnych algorytmów mierzono z dokładnością do mikrosekund. Do tego zastosowano $std :: chrono :: high_resolution_clock$.

2.1.4 Wpływ rodzaju danych na wyniki

- Wpływ gęstości grafu: Przy użyciu macierzy sąsiedztwa gęstość grafu nie zmienia teoretycznej złożoności czasowej (O). Jednak w grafach rzadszych algorytm Brute-Force zadziała szybciej w praktyce, ponieważ będzie mógł wcześnie przerywać sprawdzanie ścieżek, w których brakuje połączeń.

- Wpływ wielu ścieżek optymalnych: Istnienie kilku tras o tym samym, najmniejszym koszcie nie wpływa na czas działania algorytmów, ale znacznie zwiększa szansę, że algorytmy heurystyczne (NN, RNN, Losowy) "trafią"na jedno z rozwiązań idealnych.

- Wpływ zakresu wag krawędzi: Różnice w zakresie wag nie zmieniają złożoności obliczeniowej (czasu działania) żadnego ze zbadanych algorytmów. Wpływają jednak bezpośrednio na dokładność – przy dużej rozpiętości wag błędne decyzje algorytmów NN i RNN "kosztują"znacznie więcej, co drastycznie zwiększa błąd względny.

2.2 Wyniki eksperymentu

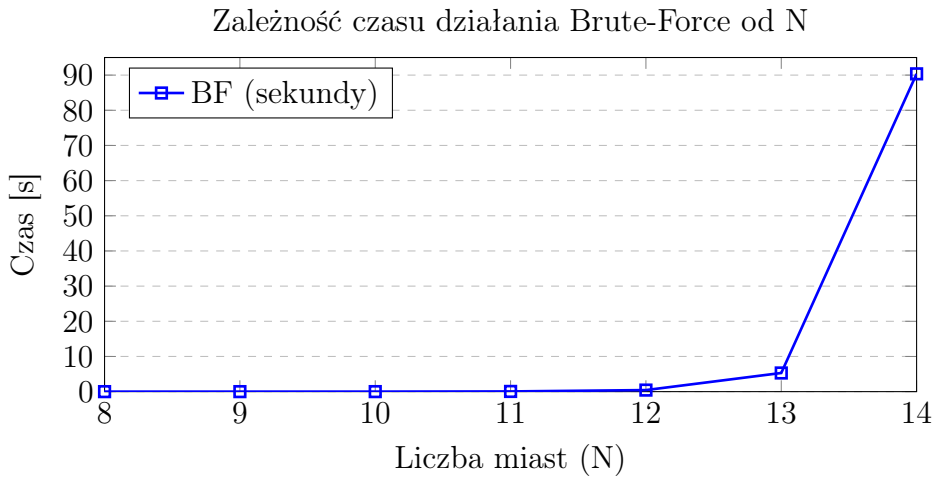
2.2.1 Zależność czasowa algorytmu Brute Force

Tabela 2.1: Pomiar czasu działania algorytmu Brute-Force (BF)

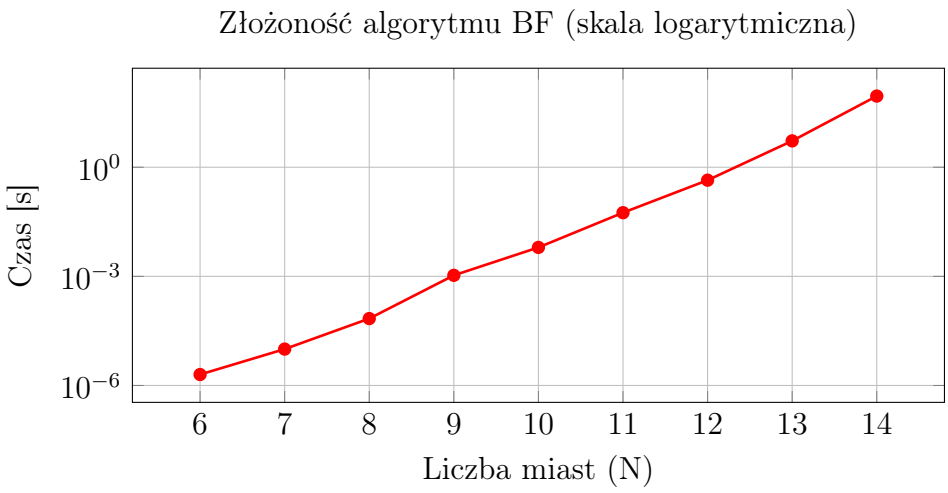
Liczba miast (N)	**Czas [μs]**	**Czas [s]**
6	2	$2 * 10^{-6}$
7	10	$1 * 10^{-5}$
8	69	$6,9 * 10^{-5}$
9	1066	$1,07 * 10^{-3}$
10	6246	$6,24 * 10^{-3}$
11	56245	0,06
12	439777	0,44
13	5302181	5,30
14	90358015	90,36
N (dla 2 min)		**$\approx$ 120**

Na tabeli widać, że złożoność czasowa jest $O(n!)$. Na tej podstawie możemy stwierdzić, że czas dla $N = 15$ będzie większy niż 2 minuty, możemy oszacować, że będzie to trwało ponad 20 minut.

Wykresy



Rysunek 2.1: Wykres czasu wykonania algorytmu BF w skali liniowej.



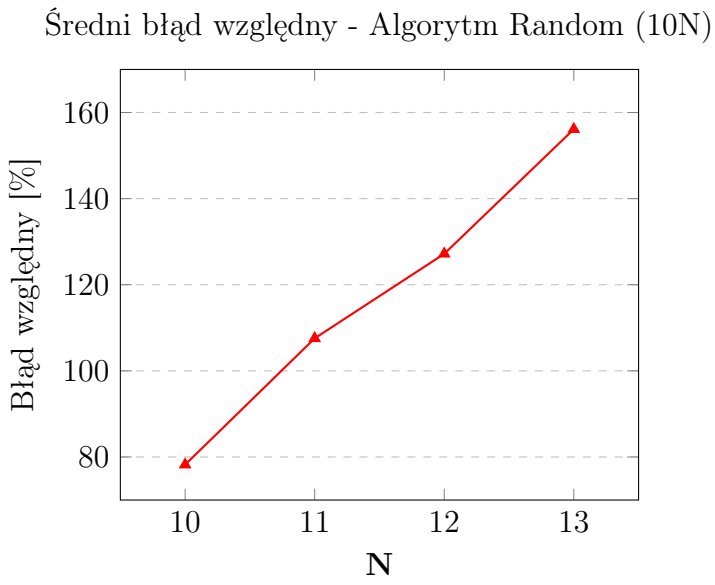
Rysunek 2.2: Wykres czasu wykonania BF obrazujący charakterystykę wzrostu czasu.

2.2.2 Średni błąd względny algorytmów

Algorytm losowy

Tabela 2.2: Średni błąd względny algorytmu Random (10N) dla 100 instancji losowych

N	Random (10N) [%]
10	78,20
11	107,55
12	127,18
13	156,12

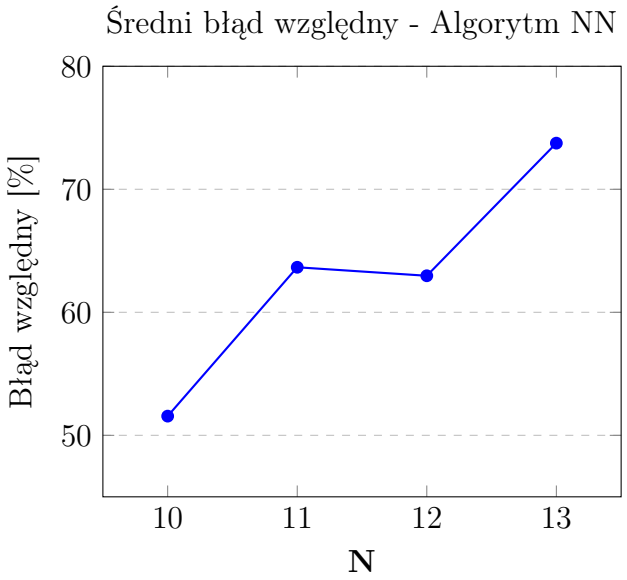


Rysunek 2.3: Wykres błędu względnego dla algorytmu Random (10N)

Algorytm NN

Tabela 2.3: Średni błąd względny algorytmu NN dla 100 instancji losowych

N	NN [%]
10	51,56
11	63,66
12	62,97
13	73,75

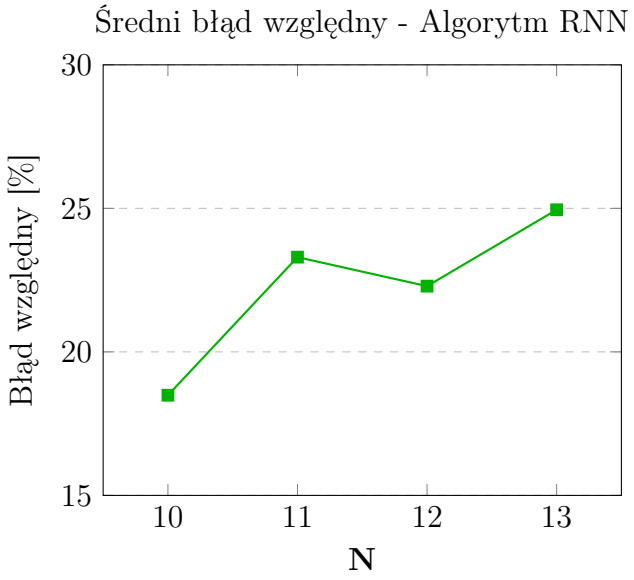


Rysunek 2.4: Wykres błędu względnego dla algorytmu NN

Algorytm RNN

Tabela 2.4: Średni błąd względny algorytmu RNN dla 100 instancji losowych

N	RNN [%]
10	18,49
11	23,30
12	22,29
13	24,95



Rysunek 2.5: Wykres błędu względnego dla algorytmu RNN

2.2.3 Średni czas algorytmów

Tabela 2.5: Średni czas wykonania algorytmów dla 100 instancji [μs]

N	**BF [μs]**	**NN [μs]**	**RNN [μs]**	**Random [μs]**
10	3760	<0.00	3	17
11	39131	<0.00	4	21
12	425984	0.02	5	25
13	8745650	0.55	12	48

3. Podsumowanie

3.1 Realizacja Projektu

Udało się pomyślnie zaimplementować wszystkie cztery zaplanowane algorytmy: dokładny Brute-Force oraz trzy algorytmy heurystyczne (Losowy, NN i RNN). Wszystkie z nich działają poprawnie, co potwierdzają zebrane i przeanalizowane wyniki pomiarów czasu oraz błędu względnego dla instancji od 10 do 13 miast. Nie udało się zdążyć zaimplementować .

3.2 Wnioski

- Algorytm Brute-Force zawsze znajduje trasę optymalną, ale jego czas działania rośnie gwałtownie, co wyklucza go z użycia dla większej liczby miast.
- Algorytm Nearest Neighbor (NN) działa błyskawicznie, jednak jego ogromną wadą jest bardzo niska dokładność i duży błąd względny.
- Algorytm Repetitive Nearest Neighbor (RNN) stanowi świetny kompromis, ponieważ drastycznie zmniejsza błąd względem NN przy wciąż bardzo krótkim czasie wykonania.
- Algorytm losowy okazał się rozwiązaniem najgorszym, dając największe błędy (nawet ponad 150%) przy czasie wykonania dłuższym niż w przypadku NN i RNN.

4. Bibiliografia

- *Algorytmy decyzyjne i teoria złożoności*, materiały dydaktyczne udostępnione przez prowadzącego przedmiot, 2026
- *Wprowadzenie do algorytmów*, T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Wydawnictwo Naukowe PWN, 2012
- *Problem komiwojażera*, artykuł w internetowej encyklopedii Wikipedia, `https://pl.wikipedia.org/wiki/Problem_komiwojaÅijera`
- *Złożoność obliczeniowa algorytmów heurystycznych*, otwarte materiały edukacyjne MIMUW Ważniak, `http://wazniak.mimuw.edu.pl`